



University of Moratuwa

Department of Electronic and Telecommunication

Assignment 01

EN4554 - Deep Learning for Vision

Sulojan.R - 220626V

Praveen V.V.J. - 220491B

Code: github.com/Sulojan2027/Vision_Coursework

4th of November 2026

1 Part 1: Does AdaLoRA's Rank Allocation Transfer to a Vision–Language Model?

1.1 Central Claim of the Paper

AdaLoRA (Zhang *et al.*, ICLR 2023) argues that the weight matrices of a pre-trained transformer are *not equally important* for fine-tuning, so a fixed parameter budget should be *allocated adaptively* rather than spread uniformly as in LoRA (Hu *et al.*, ICLR 2022). It parameterises each update in SVD form, $\Delta W = P\Lambda Q$, scores every singular triplet by a smoothed sensitivity $|w \cdot \nabla_w \mathcal{L}|$, and prunes the least important singular values until a global rank budget is met.

1.2 Concept Translated and Supporting Evidence

We test the paper's **allocation pattern**: on DeBERTaV3 (GLUE/MNLI), AdaLoRA consistently assigns **more rank to the feed-forward (FFN) matrices and to the top layers** than to attention and lower layers. This is shown by the paper's motivating experiment, where fine-tuning only FFN matrices or only top layers beats other choices at equal budget (Fig. 1 of the AdaLoRA paper), and by the final rank map of every incremental matrix (Fig. 3 of the AdaLoRA paper). The evidence is NLP-only: a VLM adds a vision tower, a connector and a language tower that *share one budget*, which raises a question that cannot be asked of a text-only model.

1.3 Translated Hypothesis

- **H1 (task–modality dependence).** With one global budget over vision encoder, connector and LM, AdaLoRA should move rank towards the modality the task needs: a larger *vision* share for text-reading (TextVQA) than for outside-knowledge questions (OK-VQA), where rank should concentrate in the LM.
- **H2 (pattern transfer).** Within each tower, the NLP pattern should reappear: FFN > attention, and top layers > bottom layers.

1.4 Experimental Setup

- **Model.** SmolVLM-256M-Instruct (256.5M parameters; SigLIP vision encoder, pixel-shuffle connector, SmolLM2 language model). It is small enough for several runs on a free Colab T4.
- **Adapted modules.** All 283 linear layers, under *one* shared budget: 72 in the vision encoder (`q,k,v,out`, `fc1,fc2` × 12 layers), 1 connector projection, and 210 in the LM (`q,k,v,o`, `gate,up,down` × 30 layers). A single budget is what lets rank move between modalities.
- **Data.** TextVQA (train / validation) and OK-VQA (train / val2014, from a parquet copy). For each task we draw **2,500 training and 500 evaluation** questions at random with seed 42. Images are resized to a 512 px longest edge. The target is the majority human answer, and the loss is masked to the answer tokens.
- **Training.** One epoch (156 optimiser steps), batch 4×4 (gradient accumulation), AdamW, lr 5×10^{-4}, cosine schedule, fp16 autocast with gradient scaling, seed 42.
- **Equal budget.** LoRA uses r=8 everywhere (α=16, 3.87M parameters). AdaLoRA starts at r=12 and prunes to an average of 8, i.e. $8 \times 283 = 2{,}264$ singular values in total. Its schedule is a 7-step warm-up, reallocation every 5 steps until step 102, then 54 fixed steps to recover (β_1=β_2=0.85, orthogonality weight 0.5). The logged budget reaches exactly 2,264 by step 100.
- **Metrics.** *Task*: VQA accuracy, $\min(\#\text{matching humans}/3, 1)$ averaged over the 10 leave-one-out annotator subsets. *Claim-tied*: the final rank of every module, aggregated by tower, module type and depth, plus the rank correlation between the two tasks' allocations. *Significance*: a paired bootstrap over the 500 evaluation questions (10,000 resamples, 95% CI).
- **Implementation note.** fp16 steps skipped by the gradient scaler carry inf/NaN gradients, which make AdaLoRA's importance scores NaN so that *nothing is ever pruned*; we zero those gradients first.

1.5 Results and Interpretation

Baseline vs adapted. LoRA improves on zero-shot by +9.3 points on TextVQA and +6.2 on OK-VQA. **AdaLoRA does *not* transfer its accuracy benefit**: at equal budget it is significantly *worse* than LoRA on both tasks (both CIs exclude 0), and on OK-VQA it barely beats zero-shot. A likely reason is that our

Table 1: VQA accuracy (%) on 500 held-out questions per task. The last row gives the paired-bootstrap difference AdaLoRA − LoRA with its 95% CI.

Method	Trainable params	TextVQA	OK-VQA
Zero-shot SmolVLM-256M	–	37.34	36.68
LoRA, uniform r=8	3.87M	**46.62**	**42.92**
AdaLoRA, 12 → 8	3.87M†	43.04	37.04
AdaLoRA − LoRA		−3.6 [−6.5, −0.7]	−5.9 [−9.1, −2.7]

†Same final number of singular values; 5.81M parameters are trained before pruning.

Colab-scale schedule (156 steps) is orders of magnitude shorter than the paper's GLUE fine-tuning. Pruning a third of the singular values within 100 steps discards directions that are still being learned. An earlier run with a later, slower pruning schedule gave the same ranking.

Table 2: Where AdaLoRA put its rank. Left: share of the total budget per tower (uniform LoRA for reference). Right: mean final rank per module (uniform = 8, maximum 12).

Tower	Uniform	TextVQA	OK-VQA	Module	TextVQA	OK-VQA
Vision	25.4%	22.0%	24.1%	Vision attention	5.1	5.7
Connector	0.4%	0.5%	0.5%	Vision FFN	10.5	11.3
LM	74.2%	77.5%	75.4%	LM attention	7.4	6.9
				LM FFN	9.6	9.7

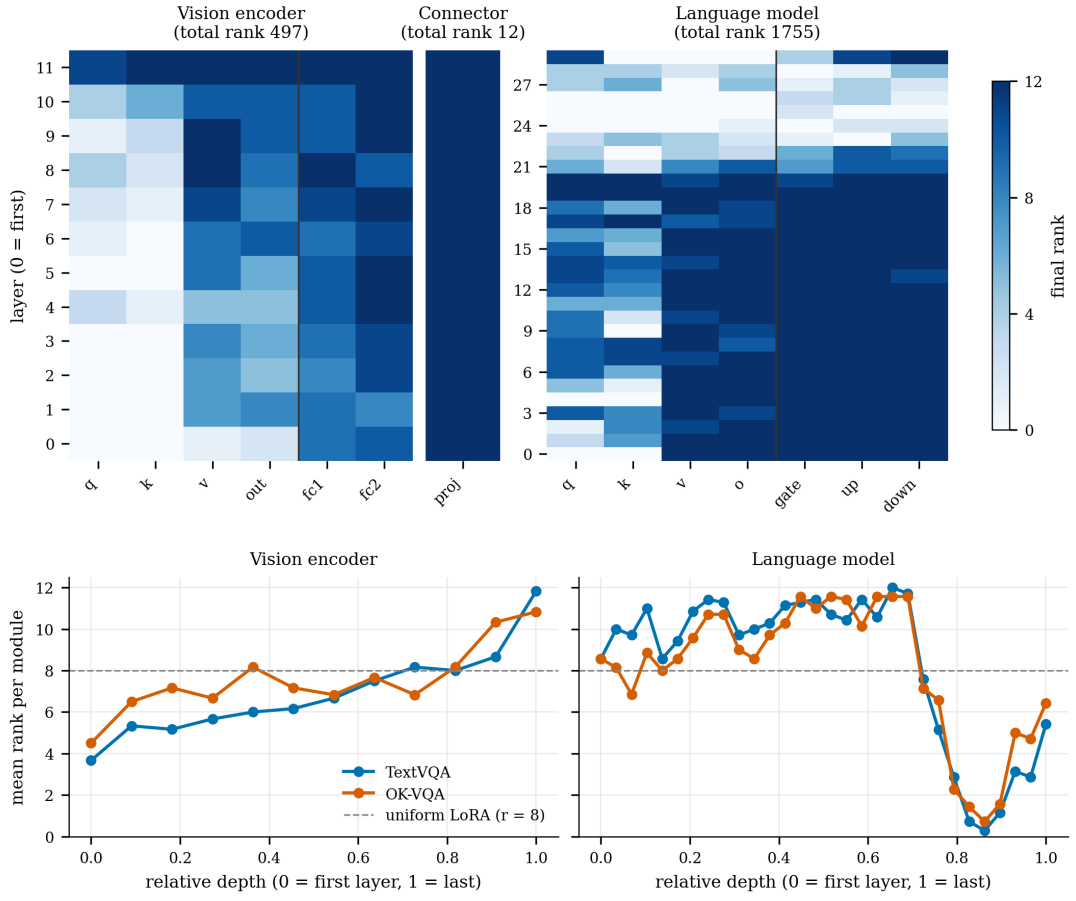


Figure 1: Top: final AdaLoRA rank per module on TextVQA (white = pruned; vertical rule separates attention from FFN). The OK-VQA map is nearly identical. **Bottom:** mean rank against relative depth for TextVQA (blue) and OK-VQA (vermillion).

H1 is rejected. The vision share is *lower* for the text-reading task (22.0% vs 24.1%), the opposite of the prediction. The two allocations are strongly correlated across the 283 modules (Spearman $\rho = 0.83$, Pearson $r = 0.89$). In this model the allocation is driven by the *architecture*, not by what the task demands from each modality. The same holds in an independent earlier run (23.2% vs 24.4%).

H2 holds partially, and the exception is vision-specific.

- *FFN > attention transfers to both towers*, most strongly in the vision encoder (10.5–11.3 vs 5.1–5.7). Query and key projections are pruned almost entirely; value, output and FFN matrices stay near full rank.
- *Top layers > bottom layers holds in the vision encoder*: mean rank rises from about 4 to 12 with depth.
- *It reverses in the language model.* Rank peaks at 40–70% depth, and layers 21–29 are pruned to almost zero. Our interpretation: the middle LM layers are where image tokens are fused with the question, so that is where adaptation pays off. The top layers only map to a short answer vocabulary.
- *The connector keeps full rank* in every run: as the only path between the towers, it is always among the most important matrices.

Bonus: is the discovered allocation useful by itself? If the allocation is informative but AdaLoRA's *training* is what hurts, then *plain* LoRA trained with AdaLoRA's final per-module ranks should beat uniform LoRA. This comes from an earlier run (same setup, slower AdaLoRA schedule: 10% warm-up, 20% final phase, reallocation every 10 steps), with the discovered ranks rescaled to the same total rank.

Table 3: Bonus, earlier run. Plain LoRA with different rank allocations (accuracy %, trainable parameters in brackets). "Swapped" trains each task with the *other* task's allocation.

Allocation	**TextVQA**	**OK-VQA**
Uniform r=8	43.80 (3.87M)	40.40 (3.87M)
AdaLoRA-discovered (same task)	**45.28** (4.20M)	**41.56** (4.31M)
AdaLoRA-discovered (other task)	45.12 (4.31M)	41.28 (4.20M)
AdaLoRA itself	43.52	37.84

The discovered allocation beats uniform on both tasks (+1.5 and +1.2) and beats AdaLoRA itself (+1.8 and +3.7). This is consistent with the allocation being useful and the pruning schedule being what costs accuracy. The other task's allocation does almost as well (within 0.3 points), which agrees with the high correlation in H1. *This result is suggestive only*, for two reasons:

- Matching total rank gave 8–11% more parameters, because the discovered allocations favour the larger FFN matrices. The notebook includes a parameter-matched version (3.84M vs 3.87M) that did not finish within our GPU budget.
- Re-running uniform LoRA with near-identical settings moved TextVQA from 43.80 to 46.62. Run-to-run variation of about 3 points is larger than these gains, and the bootstrap CIs cover only the choice of evaluation questions.

Conclusion. The NLP finding that *FFN matrices matter more* transfers to both towers of a VLM. *Top layers matter more* transfers only to the vision encoder, and *reverses* in the language model. The cross-modal split of the budget is task-invariant ($\rho = 0.83$), which rejects our task–modality hypothesis. At Colab-scale training, AdaLoRA's adaptive pruning is significantly worse than uniform LoRA. Its discovered allocation may still be useful when reused by plain LoRA. Code: `part1.ipynb` (seed 42; results, adapters, allocations, per-question scores and figures are saved by the notebook).

2 Part 2: LoRA Fine-tuning a Small LLM for Tool Calling

2.1 Setup

- **Model:** Qwen2.5-0.5B-Instruct, used through its built-in tool-calling chat template (each call is a JSON object wrapped in `<tool_call>` tags). Greedy decoding, at most 128 new tokens per turn.
- **Tools (4):** `calculator` (arithmetic), `unit_convert` (length, mass, °C/°F), `get_weather` (made-up weather for six Sri Lankan cities) and `lookup_order` (60 made-up orders). The invented values force the model to call the tools instead of answering from memory.
- **Agent loop:** generate → parse any tool call → execute it → append the result as a tool message → repeat, until a plain-text reply or 4 steps.
- **Data:** template-generated, with gold answers computed by running the tools. **Train: 1,200** requests (856 single-call, 118 two-call chains, 226 needing no tool). **Test: 100** requests (68 single-call, 5 chains, 27 no-tool; 77 unique), built from separately written templates with different wording. No test request appears in the training set.
- **LoRA:** $r = 16$, $\alpha = 32$, dropout 0.05 on the q, k, v, o, gate, up and down projections (8.80 M trainable parameters, 1.75 %). Loss on assistant tokens only; system, user and tool-result tokens are masked. 2 epochs, learning rate 2×10^{-4} (cosine, 5 warm-up steps), effective batch 16 (micro-batch 2×8 accumulation, because batch 8 ran out of memory with Qwen's 152k-token vocabulary), fp16 mixed precision, seed 42. 150 steps in 6 min 42 s on a Colab T4; training loss fell from 0.34 to 0.0003.

2.2 Baseline vs. LoRA (same 100-request test set)

Metric	Definition (denominator)	Baseline	LoRA	Change
Call validity	made ≥ 1 call, all well-formed (tool requests, $n = 73$)	83.6 %	**100.0 %**	+16.4
Call accuracy	right tool(s) and right arguments ($n = 73$)	68.5 %	**90.4 %**	+21.9
Abstention	answered without a tool when none was needed ($n = 27$)	59.3 %	**96.3 %**	+37.0
Task success	final answer contains the gold facts, end to end ($n = 100$)	55.0 %	**92.0 %**	+37.0

The baseline never produced malformed JSON: all of its validity failures were tool requests where it made no call at all.

2.3 Failure analysis

Category	Base	LoRA
Ignored tool result	15	0
Missed call	14	0
Unnecessary call	11	1
Wrong no-tool answer	7	0
Wrong tool	6	4
Wrong arguments	3	3
Wrong format	0	0
Total failures	**56**	**8**

One category per request; "missed call" includes chains cut short. On the same requests, LoRA **fixed 49** of the 56 baseline failures, **left 7 unfixed** and **introduced 1**. All 8 LoRA failures were inspected (only 8 exist, fewer than the 10 asked for), along with 10 baseline failures.

Baseline errors were decision errors, not format errors. It asked the user for data instead of calling ("should I pack a coat?" → "could you provide the current temperature?"), called tools for general knowledge (`lookup_order(order_id="RAM")` for "What does RAM stand for?"), refused easy questions ("I can only provide details about mathematical operations and weather") and misreported correct results ("the backpack is expected to be back in stock in 6 days"). **LoRA fixed** every ignored result, refusal and wrong-tool error, 10 of the 11 unnecessary calls and 10 of the 14 missed calls.
Not fixed (7): "Take b away from a" has wrong arguments in both models, and LoRA computes $b-a$ (2 cases). The boiling-point question still triggers `calculator("0*100")`, answered as "0 °C" (1). "I'm travelling to *city*, should I pack a coat?" moved from a missed call to the wrong tool, `calculator("1*24")`, answered as "24 hours until you arrive" (4). **Introduced (1):** "Vavuniya weather report, please" now calls `get_weather(city="Lisbon")`.

The remaining LoRA errors share one cause: **over-fitting to the training templates** (loss $\approx 2 \times 10^{-4}$ by epoch 2). The model keys on surface patterns, such as reading "travelling to *city*" as the trained "how many hours until … arrives" chain, and operand order in subtraction, rather than on intent. More varied phrasings, paraphrase augmentation or a single epoch would be the next step. The ignored-result count is also somewhat inflated by a strict answer check that requires both the order status and the item.

2.4 Would a better prompt alone have been enough?

Only partly: the baseline already produced well-formed calls through Qwen's built-in tool template, so its failures were decisions (whether to call a tool at all in 32 cases, how to report the result in 15), and a stricter system prompt with a few worked examples could plausibly reduce these without any training. However, a 0.5 B model follows long instructions unreliably, and LoRA removed 49 of the 56 failures after under seven minutes of training (we did not run a prompt-only ablation to measure the gap). The errors LoRA left come from template over-fitting, which better training data, not a better prompt, would address.